

05 — Engine Systems

Three runtime systems sit between “a verb runs once when typed” and “the world has ongoing, reactive behavior”: **hooks** (verbs that fire on engine lifecycle events), the **ticker** (verbs that fire on a schedule), and **effects** (a higher-level timed buff/debuff layer built on the ticker). All three are available to any verb author; none require engine changes.

- [Hooks](#)
- [The ticker system](#)
- [The effects system](#)

Hooks

A **hook** is a named lifecycle event the engine fires while performing an action — moving an object, recycling it, a player connecting. Verb code can intercept the event by defining a verb with the right name on the affected object (or any ancestor). For *cancellable* hooks, the verb can return `False` to veto the action. The system lives in `moo/hooks.py`.

How resolution works

Each hook point maps a logical name to a list of **verb-name aliases**. When the engine calls `fire_hook("before_move", obj, args)`, it searches `obj` and its inheritance chain for a verb matching any of that hook’s aliases; the first match runs. If none exists, nothing happens (hooks are opt-in). So to react to an event, you just add a verb with one of the alias names to the object that should react.

```
fire_hook(hook_point, obj, args_str='') # returns the verb's result (or None)
```

For a cancellable hook, `fire_hook` returns `False` when the hook verb returned `False`, and the engine aborts the pending action.

The hook registry

These hook points are registered at startup. The **alias** column lists the verb names you can define to handle the event; the first is canonical.

Hook	Fired on	Cancellable	Aliases (verb names you define)
<code>before_move</code>	the mover	✓	<code>at_before_move</code> , <code>at_pre_move</code>
<code>before_leave</code>	old location	✓	<code>at_before_leave</code>
<code>before_enter</code>	destination	✓	<code>at_before_enter</code>
<code>after_move</code>	the mover	—	<code>at_after_move</code> , <code>at_post_move</code>
<code>after_leave</code>	old location	—	<code>at_after_leave</code> , <code>exit_func</code>
<code>after_enter</code>	destination	—	<code>at_after_enter</code> , <code>enter_func</code>
<code>object_creation</code>	the new object	—	<code>at_object_creation</code>
<code>before_recycle</code>	object being deleted	✓	<code>at_before_recycle</code>
<code>object_delete</code>	location of the deleted object	—	<code>at_object_delete</code>
<code>before_reparent</code>	object being reparented	✓	<code>at_before_reparent</code> , <code>at_pre_reparent</code>
<code>after_reparent</code>	object after reparenting	—	<code>at_after_reparent</code> , <code>at_post_reparent</code> (args = old parent #)
<code>on_disconnect</code>	player object	—	<code>at_disconnect</code> , <code>on_disconnect</code>
<code>on_puppet</code>	character	—	<code>at_puppet</code> , <code>on_puppet</code> (fires after move to <code>last_location</code>)
<code>on_unpuppet</code>	character	—	<code>at_unpuppet</code> , <code>on_unpuppet</code> (fires before storage in #2)

Note the `enter_func` / `exit_func` aliases on `after_enter` / `after_leave` — that’s how containers and rooms run arrival/departure logic. The `at_post_move` alias on `after_move` is the standard “auto-look on arrival” hook (defined on #3 so every character inherits it).

A movement is a sequence of hooks

`move()` doesn’t fire one hook — it fires a sequence, any of whose *before* hooks can cancel the whole move:

1. `before_move` on the mover (cancellable)

2. `before_leave` on the old location (cancellable)
3. `before_enter` on the destination (cancellable)
4. — the move happens —
5. `after_move` on the mover, `after_leave` on the old location, `after_enter` on the destination

Examples

Block movement while a condition holds (cancellable before-hook):

```
# verb 'at_before_move' on a character
if getattr(pobj, 'rooted', 0):
    pObj.msg("You can't move – your feet are rooted to the floor!")
    result = False      # cancels the move
```

React after a player arrives (informational after-hook):

```
# verb 'at_after_enter' (or 'enter_func') on a room
this.msg_room("%S arrives.", exclude=[pobj], sub=pobj)
```

Registering your own hook

Game code can add hook points at runtime:

```
register_hook(name, aliases, cancellable=False, description="")
list_hooks()           # all registered hooks + metadata
is_cancellable(name)   # True for before-hooks
```

`register_hook("before_say", ["at_before_say"], cancellable=True, ...)` then lets any object veto or rewrite speech by defining `at_before_say`.

The ticker system

The ticker is the engine's heartbeat: "call this verb on this object every N seconds." It is the timing backbone for the effects system, round-time countdowns, NPC behavior loops, and any recurring behavior. It lives in `moo/ticker.py`.

API

```
ticker_add(interval, verb_name, obj, idstring='') # subscribe
ticker_remove(obj, idstring='')                 # remove one subscription
ticker_remove_all(obj)                           # remove all for an object
ticker_list(obj=None)                             # list subscriptions
```

Parameter	Meaning
<code>interval</code>	Seconds between fires (float).
<code>verb_name</code>	The verb to call on <code>obj</code> each tick.
<code>obj</code>	Target object (a <code>M000bject</code> or an objnum).
<code>idstring</code>	Unique label for this subscription on that object.

The `idstring` must be unique per object; re-adding with the same object + `idstring` **replaces** the existing subscription. Conventionally it embeds the objnum, e.g.:

```
ticker_add(1, '_td_rt', pobj, f'_td_rt_{pobj.objnum}') # tick down round-time every
second
```

Resolution and threading

The ticker has **1-second resolution**: the event loop checks once per second for due subscriptions. Tick callbacks run on the **same single verb-execution worker** as player commands (see Architecture), so they never run concurrently with a command or with each other — no locking required inside tick verbs.

Persistence

Subscriptions are stored in the `tickers` **table of the SQLite database**, so they survive restarts. On startup every subscription is reloaded and its `next_fire` is recalculated from the current time. (The older one-file-per-object builds wrote a `tickers.json`; the current SQLite engine uses the table.)

Self-cancelling tickers

A common pattern: a tick verb that counts a property down and removes its own subscription when it reaches zero. The round-time / status countdown machinery on `#1` drives many timers (`rt`, and status/condition counters) through a single configurable tick-down verb, each removing itself when it expires — so idle characters carry no live tickers.

The effects system

Effects are timed, repeating, optionally-stacking modifiers — poison ticking damage, regeneration restoring it, intoxication wearing off — built on top of the ticker. The whole system is one utility object: `#53`, aliased `$eu` (and also injected as `eu`). Source: `moo/effects.py`, with the callable verbs in `moo verbs/53/`.

API

```
eu.trigger(pobj, name, ticks, interval, *args, **kwargs)
eu.trigger_all(pobj, effects_list)
eu.cancel(pobj, name=None)
eu.list_active(pobj)
```

Method	Purpose
<code>trigger(pobj, name, ticks, interval, *args, **kwargs)</code>	Apply an effect that fires <code>ticks</code> times, <code>interval</code> seconds apart. Extra args are forwarded to the callback.
<code>trigger_all(pobj, effects_list)</code>	Apply several at once; each item is <code>(name, ticks, interval, *extra)</code> .
<code>cancel(pobj, name=None)</code>	Cancel one named effect, or all effects on <code>pobj</code> if <code>name</code> is omitted.
<code>list_active(pobj)</code>	Return the active effects on <code>pobj</code> (<code>name</code> , <code>remaining</code> , <code>tick</code> , <code>interval</code>).

```
eu.trigger(pobj, "poison", 5, 3, damage=10)           # 5 ticks, every 3s, 10 dmg each
eu.trigger_all(pobj, [("poison", 5, 3, 10), ("regen", 10, 6)])
```

Effect callbacks

Each effect named `X` is implemented by a verb `do_X` on #53. When the effect fires, the engine calls `do_X` with:

Variable	Meaning
<code>pobj</code>	The target the effect is on.
<code>tick</code>	Current fire count (1-based).
<code>remaining</code>	Ticks left after this fire.

```
# do_intoxicate on #53
if tick == 1:
    pobj.msg("Your vision begins to blur...")
pobj.msg("The world sways around you." if remaining else "Your head begins to clear.")
```

Adding a new effect type is therefore just: write a `do_<name>` verb on #53, then `eu.trigger(pobj, "<name>", ...)` from anywhere.

Stacking and persistence

Calling `trigger` again with the same target, name, interval, and args **stacks** — it adds to the remaining ticks rather than creating a duplicate. Active effects live in the `fx_registry` property on #53, so (being a property on a database object) they persist across restarts.

The dispatcher

A hidden `_tick` verb on #53 is registered as a 1-second ticker. Each second it scans `fx_registry`, fires every effect whose `next_fire` is due, and — when the registry empties — removes its own ticker so an idle world spends no cycles on effects. This is the effects system eating its own dog food: it's just a ticker plus a dict.

Next: The Prototype Library — the shippable base objects (rooms, exits, containers, furniture, wearables, consumables) and their properties.